

Relatório - Mini E-commerce Distribuído

Disciplina: Fundamentos de Computação e Programação Distribuída

Projeto: Mini-E-commerce-Distribuído

Tecnologias: JavaScript, Node.js, Express, JWT, bcryptjs, axios, JSON, Docker Compose

Introdução

Este relatório descreve a implementação de um mini e-commerce distribuído composto por API Gateway (porta 5000), Users Service (5001), Products Service (5002) e Orders Service (5003). O cliente acessa exclusivamente o Gateway, que encaminha requisições HTTP/REST aos microsserviços internos. O sistema incorpora heartbeat para tolerância a falhas, sincronização entre réplicas de produtos e execução via Docker Compose.

1. Comunicação entre os microsserviços

A comunicação foi implementada por HTTP/REST. O cliente acessa apenas o API Gateway na porta 5000. O Gateway encaminha as chamadas para os serviços internos usando axios, preservando método HTTP, corpo JSON e header Authorization. Os microsserviços são independentes e executam em portas separadas.

No Docker Compose, a comunicação interna utiliza os nomes dos containers: `http://users:5001`, `http://products:5002` e `http://orders:5003`. Cada serviço valida JWT localmente quando necessário, sem consulta síncrona ao Users Service em cada operação.

2. Estratégia de consistência na replicação

O Products Service utiliza duas réplicas JSON: `products_replica_1.json` e `products_replica_2.json`. A estratégia adotada é consistência forte simplificada: toda escrita de produto é realizada nas duas réplicas antes de confirmar sucesso ao cliente. Se uma das réplicas falhar na escrita, o serviço não confirma sucesso e retorna erro claro.

As leituras são distribuídas por round-robin entre as réplicas. Além disso, implementamos uma rotina de sincronização que compara as duas réplicas, unifica produtos pelo campo id e corrige divergências simples. A sincronização é executada ao iniciar o serviço e antes de leituras quando inconsistência é detectada.

Não se trata de replicação robusta de produção, mas atende ao objetivo didático de demonstrar escrita replicada, leitura distribuída e recuperação de inconsistências locais.

3. Falha no Serviço de Pedidos

Se o Orders Service cair, o Gateway detecta a falha por heartbeat. A cada 5 segundos, envia GET /health com timeout de 2 segundos. Após 2 falhas consecutivas, marca o serviço como indisponível, registra log com timestamp e quantidade de falhas, e as rotas de pedidos retornam:

```
{  
  "error": "Serviço indisponível no momento"  
}
```

com status HTTP 503 Service Unavailable.

Os serviços Users e Products continuam funcionando normalmente, pois são microsserviços independentes. O Gateway bloqueia apenas rotas do serviço indisponível. Quando Orders volta a responder, o Gateway detecta recuperação, atualiza lastRecoveryAt no endpoint GET /status, zera o contador de falhas e libera as rotas novamente. A recuperação é didática: não há sincronização de dados perdidos durante a indisponibilidade, pois pedidos não foram processados enquanto o serviço estava fora.

4. Segurança com JWT e controle de admin

O login no Users Service gera JWT assinado com JWT_SECRET, contendo userId, email, role e exp. A senha é armazenada com hash bcryptjs.

O Products Service valida JWT e verifica role === "admin" antes de POST /products. Usuário comum recebe 403 Forbidden. No Users e Orders Services, usuário comum acessa apenas seus próprios dados; admin acessa qualquer usuário ou pedido.

5. Limitações em relação a um sistema real

- Uso de arquivos JSON em vez de banco de dados transacional.
- Recuperação didática: sem mensageria, fila persistente, transações distribuídas nem consenso real.
- Em produção seria necessário banco de dados real, logs persistentes, retry/backoff, observabilidade e mecanismos robustos de recuperação.
- Replicação simples e local, sem réplicas em servidores distintos.
- Ausência de rollback automático em cenários complexos de falha parcial.
- Ausência de HTTPS/TLS, autenticação mútua entre microsserviços e fila de mensagens.
- Logs simples no terminal, sem agregação centralizada.
- Estoque não é reduzido automaticamente ao criar pedido.
- Orders Service não valida existência do produto antes de criar pedido.
- Variáveis sensíveis simplificadas (JWT_SECRET=supersecret).

Testes realizados

1. Subida com docker compose up --build.
2. GET /health e GET /status no Gateway.
3. Cadastro e login de admin e usuário comum.
4. Criação de produto com token admin; tentativa de usuário comum retornando 403.
5. Criação e listagem de pedidos.
6. docker compose stop orders — Gateway retorna 503 em rotas de pedidos.
7. docker compose start orders — Gateway registra recuperação em log e /status.
8. Consultas repetidas a produtos alternando entre réplicas com dados sincronizados.

Conclusão

O projeto demonstra decomposição em microsserviços, comunicação REST, API Gateway, autenticação JWT, replicação simplificada com sincronização, tolerância a falhas com heartbeat e execução via Docker Compose. As limitações refletem o caráter didático e indicam evoluções necessárias para produção.